代码说明 • 技术设计 • 复现验证

telescope_gpu 工程化技术说明

C++ 数值库 · CCD 管线 · 学习型相位反演 · 测试与部署

代码仓库：github.com/Starry-Journey/telescope_gpu

分支：engineering-mvp

对齐文档：《大口径望远镜重力变形标定技术研究（代码对齐版）》

运行环境：Ubuntu 20.04 / WSL2，CPU + CUDA（原项目）

版本：v1.0 • 2026-11

> *本文的数值分为三类，不能混用：① 主技术报告的相位差反演结果（256²、15 星、5 帧）；② CCD 管线的仿真检测结果；③ 学习型反演的 64×64 PSF 方法验证。每张表都标注口径。*

00 文档范围与阅读顺序

> 先明确哪些是主求解器、哪些是工程扩展，再看后面章节。

本文不是“项目介绍”，而是 **代码与主技术报告对齐后的技术说明**：说明每个模块的科学原理、软件设计、函数职责、代码注释位置、测试方法与结果。所有数字都给出运行口径。

表 0-1　阅读地图

章	内容	对应代码/证据
01	代码地图、与主技术报告的对齐关系	全仓库 / 报告表 3-1、4-1、4-2
02	系统架构、分层、数据流	CMake、include/、src/、pipeline/
03	科学原理：相位差、Zernike、图像域 SSE、CCD/PSF	zernike.cpp、phase_diversity.cu
04	C++ 数值库：LU/QR/SVD/FFT/RK4、GTest、Benchmark、pybind11	numerics/
05	CCD 管线：定标、星点、帧差/Hough、WCS、评估	pipeline/
06	学习型反演：可微前向、CNN、two-stage	ml/
07	FastAPI + React 科学计算小程序	web/
08	测试矩阵、CI、性能与失败模式	ctest、.github/workflows/ci.yml
09	运行说明、输出文件、模块注释索引	README、docs/代码模块注释索引.md

三个口径

主报告：0.001667 λ 是 **波前 RMS**（已知真值仿真）；CCD 管线：0.85 是 **目标检测率**；学习型反演：34/1823 是 **BOBYQA 评估次数**。三个数字回答不同问题，不能放在一张“精度表”里比较。

01 代码地图与主技术报告对齐

> 这一章先回答“新代码和报告是什么关系”。

1.1 代码地图

表 1-1　代码地图

目录/文件	职责	输入 → 输出
include/telescope/、src/	主项目的 Zernike/FFT/CPU 参考/FITS/配置/GPU 正向模型/相位差求解	配置/星场 → 5 帧 FITS/系数
apps/	forward_generate、inverse_solve 两个 CLI	配置 → FITS/报告
tests/	自建轻量测试框架 + CPU/GPU 测试	断言 → CTest
pipeline/	CCD 定标/星点/帧差/Hough/WCS/评估	raw FITS → 星表/轨迹/指标
numerics/	Eigen 数值库：LU/QR/SVD/FFT/RK4 + GTest/Benchmark/pybind11	矩阵/向量 → 解/频谱/轨迹
ml/	PyTorch 可微前向 + CNN + two-stage	5 帧 PSF → Zernike 系数
web/	FastAPI + React/Plotly 小程序	参数 → PSF/反演结果
docs/	技术说明、教程、图、模块索引	—

1.2 与主技术报告的对齐表

主报告的核心链路是：**5 帧已知离焦星场 → 图像域 SSE → 40 起点 + 坐标下降 + 前 3 名 BOBYQA → Z4/Z8/Z9 → 3×5 灵敏度矩阵/补偿**。本工程的主求解器仍然是 src/phase_diversity.cu；numerics/ 和 ml/ 是工程扩展，不替代主求解器。

表 1-2　代码与报告逐项对齐

报告条目	报告口径	本工程对应	对齐/差异
离焦帧	5 帧：0、±1、±2 λ	`configs/default.conf`；`ml` 的 5 帧顺序	一致
反演变量	Z4/Z8/Z9（3 项）	主求解器；`ml` 输出 3 项	一致
代价函数	图像域归一化 SSE	`phase_diversity.cu` 的 `cost`	一致
优化	40 起点 + 坐标下降 + 前 3 名 BOBYQA	`solve_starts=40`、`solve_refine=3`	一致
BOBYQA	`maxeval=2000`、ftol=1e-10、xtol=1e-6	`configs/default.conf`	一致
灵敏度矩阵	3×5，外部 Zemax 标定，代码未实现	报告表 2-6；代码无 IM	一致（有意留空）
主结果	45° 波前 RMS 0.001667 λ，5692 次评估	报告表 4-1；主程序复现	一致
成像规模	256×256、过采样 2、15 星	主程序默认配置	一致
CCD 管线	报告未展开	`pipeline/` 独立扩展	补充，不混结果
学习型反演	报告未展开	`ml/` 64×64 PSF 代理	方法验证，不替代主求解器

> **不要混表**
>
> `ml` 的 64×64 PSF 模型没有星点叠加、没有 15 颗星、没有 256²/过采样 2；它的 0.00955 波前 RMS 是“方法验证”的数字，不能写成主报告 0.001667 λ 的复现。主报告结果只以主程序/报告表 4-1 为准。

1.3 本文的边界

- 主报告解决科学问题：重力变形 → 低阶像差 → 相位差反演；本文解决工程问题：数值库、CCD 前端、学习型代理、部署。
- `numerics/` 是可复用数值核心；`pipeline/` 是 CCD 数据前端；`ml/` 是学习型反演实验；三者独立构建/测试。
- 所有仿真结果都不是 2.5 m 望远镜实测；真实数据需要重新标定。

02 系统架构与数据流

从“一帧 FITS”到“系数/指标/页面”的分层。

2.1 分层

表 2-1　分层与依赖

层	目标	依赖	构建开关
telescope_cpu	Zernike/FFT/FITS/配置/CPU 参考；无 GPU 可编译	C++17、std::filesystem	总是构建
telescope_cuda	GPU 正向模型 + 相位差求解	CUDA/cuFFT/NLopt	TEL_BUILD_CUDA=ON
telescope_numerics	Eigen 数值库 + GTest/Benchmark/pybind11	Eigen、GTest、Benchmark、pybind11	TEL_BUILD_NUMERICS=ON 或独立构建
pipeline	CCD 数据前端与评估	numpy/scipy/astropy/matplotlib	Python
ml	可微前向、CNN、two-stage	PyTorch、NLopt	Python
web	FastAPI + React 小程序	FastAPI/uvicorn/torch/nlopt/Plotly CDN	Python + 浏览器

2.2 主项目数据流

代码 2-1　主项目的正向/反演数据流（与报告 3.3/3.4 一致）。

```
forward_generate
  make_aberration_model(theta)            # 报告 2.2.3 的重力变形参数化
  ForwardModelGPU::generate_frames(c)     # 光瞳 -> cuFFT -> PSF -> 降采样 -> 星点
  Poisson + read noise -> FITS            # 5 帧 + 星表头 + 真值

inverse_solve
  load_frame_set -> preprocess_frames     # 扣背景、逐帧归一化
  PhaseDiversitySolver::solve
    40 starts -> coordinate descent
    top 3 -> NLopt BOBYQA (maxeval=2000)
  inverse_result.txt / inverse_report.txt
```

2.3 CCD 管线数据流

代码 2-2　`pipeline/` 的五段数据流。

```
raw FITS + bias/dark/flat
  -> preprocess (background/denoise)
  -> detect_stars (threshold/label/centroid)
  -> detect_moving (registration/median/difference/morphology/Hough/tracks)
  -> solve_astrometry (TAN WCS + cubic distortion + catalogue match)
  -> evaluate (detection rate / false alarm / position RMSE / sky RMSE)
```

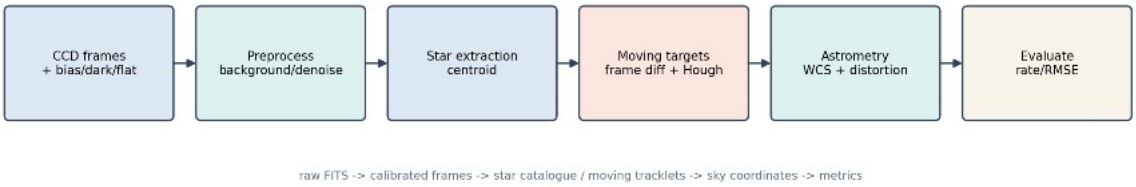


图 2-1　CCD 管线的六个阶段。每个阶段有独立输入输出，可替换实现。

2.4 学习型反演数据流

代码 2-3　可微前向 + CNN + BOBYQA 的 two-stage。

```
coeff (c4,c8,c9)
  -> torch pupil + 5 defocus phases
  -> torch.fft2 -> |.|^2 -> 2x2 binning
  -> CNN -> initial coeff
  -> NLopt BOBYQA (NumPy forward) -> refined coeff
```

主项目的正向模型与 `ml` 的正向模型是两套实现：前者 256²/过采样 2/15 星/星点叠加，后者 64×64/纯 PSF/无星点。它们共享“Zernike + 离焦 + 傅里叶传播”的物理，但数值不能直接互比。

03 科学原理

> 这一章只讲主链路和扩展模块共用的物理/数学，不重复报告全文。

3.1 相位差法的成像模型

光瞳复振幅为 P(u,v)=A(u,v)·exp[i2π φ(u,v)]，焦面复振幅是傅里叶变换 U=F{P}，PSF=|U|²。对第 i 帧，在光瞳上加已知离焦 d_i：

$$\phi_i(u,v) = \Sigma_k\, c_k\, Z_k(u,v) + d_i \cdot Z4(u,v)$$

5 帧 d_i = 0、±1、±2λ 提供相位多样性；同一组 c 必须同时解释 5 帧。主项目用 8 个 Zernike 槽位，但只拟合 Z4/Z8/Z9；其余置零。

3.2 Zernike 与编号

Z4 是离焦，Z8/Z9 是 x/y 彗差。必须注意：本项目的 Z9 **不是** Noll Z9；Noll Z9 是 trefoil。工程标签与 Noll 的对应见报告表 C-1 和 `docs/代码模块注释索引.md`。

表 3-1　工程标签与 Noll 编号（与报告附录 C 一致）

工程标签	物理含义	真实 Noll	是否拟合
Z4	离焦	Noll Z4	是
Z5	45° 像散	Noll Z5	否
Z6	0° 像散	Noll Z6	否
Z8	彗差 x	Noll Z8	是
Z9	彗差 y	Noll Z7	是

3.3 图像域 SSE 与优化

主项目代价函数：

$$J(c) = \Sigma_i\, \Sigma_p\, [\, d_i(p) - m_i(p;c)\,]^2$$

其中 d_i 是预处理后的实测/仿真帧，m_i 是模型帧。主报告使用 **逐帧归一化** 后的图像域 SSE；不是 Gonsalves 频域消元。优化流程：

- 40 个随机起点（对称边界，允许负 Z9）；
- 坐标下降粗搜，步长 0.3 开始，不改进就减半，直到 1e-4；

- 前 3 名送 NLopt BOBYQA，`maxeval=2000`，`ftol=1e-10`，`xtol=1e-6`；
- 诊断：`cost(true)` 是模型自洽下界，`cost(zero)` 衡量非线性；最终代价低于 true 超过 1% 报警。

3.4 主报告结果口径

表 3-2　主报告多俯仰角结果（来自报告表 4-2；仿真）

θ	真值 Z4/Z8/Z9	反演 Z4/Z8/Z9	波前 RMS	评估次数
0°	2.0000/0.2300/0.1331	2.0007/0.2312/0.1336	0.00151	5292
45°	1.4728/0.1714/0.1800	1.4729/0.1723/0.1786	0.00167	5692
90°	0.2000/0.0300/0.1331	0.1984/0.0293/0.1318	0.00222	9587
270°	0.2000/0.0300/−0.0931	0.1984/0.0311/−0.0931	0.00192	8836
315°	−1.0728/−0.1114/−0.1400	−1.0725/−0.1112/−0.1381	0.00199	7054

45° 的逐项误差：Z4 0.000130 λ、Z8 0.000856 λ、Z9 0.001423 λ；反演代价 5.3124e-3，真值代价 5.3125e-3，差 0.0018%。这些数字只属于主求解器。

3.5 CCD/PSF 与噪声（扩展模块）

CCD 管线把探测器响应写成 raw=(S+B+D·t)·F+bias+noise；定标为 cal=(raw−bias−D·t)/F。星点用高斯 PSF 近似，σ=1.6 pixel；噪声含泊松+读出 5 e− RMS。学习型反演用 64×64 纯 PSF，不含星点叠加，因此只能做方法验证。

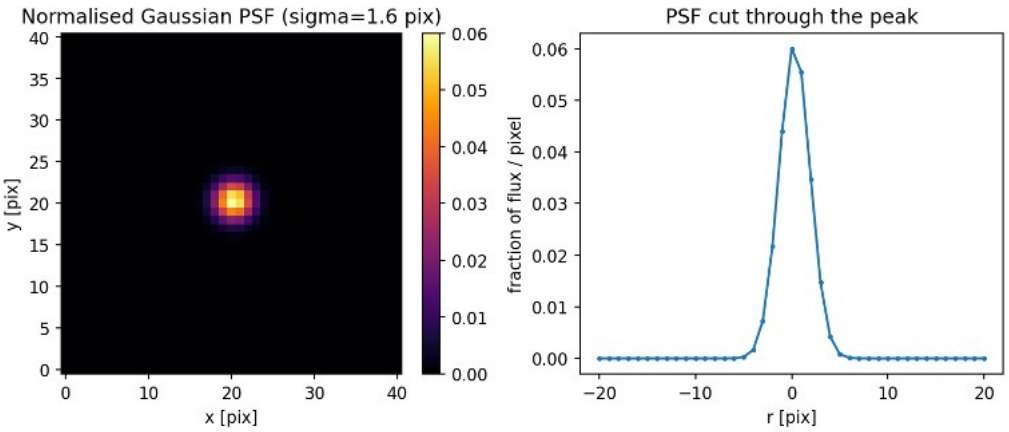


图 3-1　高斯 PSF 的二维形状与过峰截面（σ=1.6 pixel，总流量归一化）。

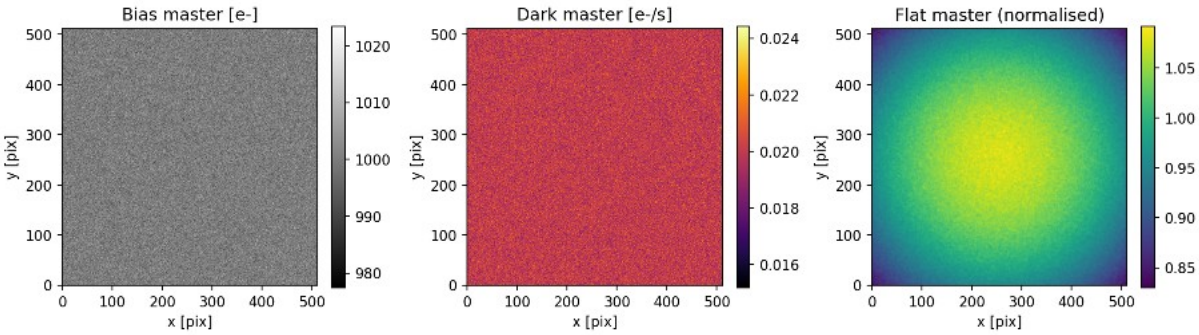


图 3-2　bias/dark/flat 三个 master 帧：偏置、暗电流、平场响应。

04 C++ 数值库设计与测试

> Eigen + GTest + Google Benchmark + pybind11，每个函数都能被独立验证。

4.1 模块与 API

表 4-1　数值库模块

文件	职责	关键接口	注释位置
numerics.hpp	公开 API：解方程、最小二乘、FFT、RK4	solve_linear、least_squares、fft、solve_ode_*	文件头 + 函数前 Doxygen
numerics.cpp	Eigen 实现 + 异常/边界检查	同上	算法步骤注释
tests/test_numerics.cpp	10 个 GTest	解析解/NumPy/Eigen 对照	每个 TEST 前说明目的
bench/bench_numerics.cpp	4 组 Google Benchmark	solve/lstsq/FFT/RK4	基准参数注释
bindings/numerics_py.cpp	pybind11 模块	telescope_numerics_py	模块 docstring

代码 4-1　六个公开接口：单位/异常在头文件注释中。

```
#include "telescope/numerics.hpp"
using namespace telescope::numerics;

Vector x = solve_linear(A, b);              // LU, partial pivot
Vector c = least_squares(A, b);             // QR
Vector p = fit_polynomial(xs, ys, 2);       // polynomial least squares
std::vector<Complex> X = fft_copy(x);      // radix-2
OdeResult r = solve_ode_scalar(rhs, y0, t0, t1, dt);
```

4.2 算法与复杂度

表 4-2　算法/复杂度/失败模式

算法	做法	复杂度	失败模式
线性方程组	部分主元 LU	$O(N^3)$	非方阵/奇异 → 异常
最小二乘	列主元 Householder QR	$O(MN^2)$	秩亏 → 用 SVD 版本
最小二乘 SVD	Thin SVD	更慢	病态/秩亏更稳
FFT	radix-2 Cooley–Tukey	O(N log N)	非 2 的幂 → 异常
RK4	经典四阶	O(steps)	负步长/倒序时间 → 异常

为什么不用正规方程？A^TA 会把条件数平方，舍入放大；QR/SVD 更稳。为什么 FFT 只支持 2 的幂？radix-2 分治要求；非 2 的幂可补零/Bluestein，但不在本库范围。

4.3 GTest：10 项

表 4-3　数值库 GTest（10/10 通过）

测试	验证内容	判据
LinearSolve.Known3x3	已知 3×3 解	残差 <1e-12，解误差 <1e-10
LinearSolve.RejectsSingularAndBadShape	奇异/非方阵	抛异常
LeastSquares.LineFit	直线拟合	斜率/截距误差 <0.02
LeastSquares.SvdMatchesQrForWellConditioned	QR vs SVD	解差 <1e-10
Polynomial.FitQuadratic	二次多项式	系数误差 <1e-9
Fft.ImpulseAndRoundTrip	冲激谱 + 逆变换	误差 <1e-12
Fft.MatchesNaiveDft	32 点朴素 DFT	误差 <1e-10
Ode.ExponentialScalar	y'=y	与 e^1 误差 <1e-7
Ode.HarmonicOscillatorSystem	谐振子	2π 后回初值，误差 <2e-6
Ode.RejectsBadStep	dt=0	抛异常

4.4 Benchmark

表 4-4　Google Benchmark 结果（只作量级参考）

基准	规模	时间（本机 CPU）	拟合
BM_SolveLinear	8×8	~0.50 µs	—
BM_SolveLinear	32×32	~11.8 µs	—
BM_SolveLinear	128×128	~0.57 ms	0.27 N^3
BM_LeastSquares	256×16	~21 µs	—
BM_LeastSquares	1024×64	~0.69 ms	—
BM_Fft	256	~3.2 µs	—
BM_Fft	1024	~13.8 µs	—
BM_Fft	4096	~67 µs	1.36 N log N
BM_Rk4Scalar	1000 steps	~14 µs	—

4.5 pybind11

代码 4-2　Python 侧调用：Eigen 类型经 pybind11/eigen.h 转 NumPy。

```
import numpy as np
import telescope_numerics_py as tn

A = np.array([[4., -2., 1.], [-2., 4., -2.], [1., -2., 4.]])
b = np.array([11., -16., 17.])
print(tn.solve_linear(A, b))       # [1, -2, 3]
print(tn.fft([1+0j, 0j, 0j, 0j]))  # [1,1,1,1]
```

构建要点：静态库必须 `POSITION_INDEPENDENT_CODE ON`，否则共享模块链接报 relocation；Python ABI 必须与编译时一致（本机 `/usr/bin/python3` 3.8）。

05 CCD 管线模块

从原始帧到星表、运动目标、天球坐标；每个函数都有明确的输入输出。

5.1 模块职责

表 5-1　CCD 管线模块

模块	关键函数	输入	输出
simulate.py	simulate_dataset、make_wcs、distort_pixels	SimConfig	raw FITS、bias/dark/flat、catalog/truth CSV
preprocess.py	calibrate、estimate_background、denoise	raw + master	calibrated FITS/NPZ
detect.py	detect_stars、phase_cross_correlation、hough_lines、detect_moving	帧栈	星表、候选、轨迹
astrometry.py	DistortionModel、match_sky、solve_astrometry	星表 + WCS	畸变系数、RA/Dec、残差
evaluate.py	match_by_frame、star_metrics、evaluate	真值 + 检测	metrics.json/CSV
visualize.py	fig_dataset、fig_moving、fig_metrics	结果	PNG

5.2 预处理

定标公式与主报告一致：cal=(raw−bias−D • t)/F；背景用 64×64 mesh 的 sigma-clip 中值，再三次插值；去噪用 σ=0.8 的高斯，低于 PSF σ=1.6。

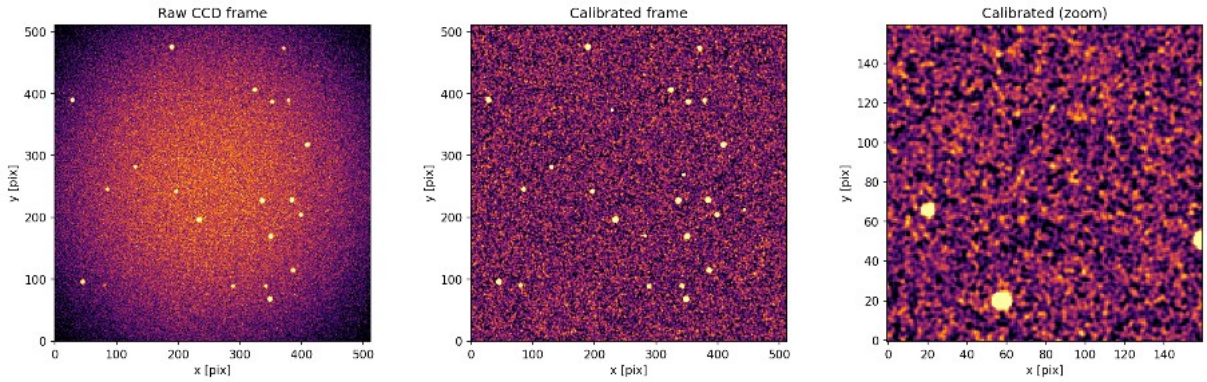


图 5-1　原始帧、定标+背景扣除帧、中心放大。原始帧橙色是 300 e− 天空背景。

5.3 星点提取与质心

代码 5-1　阈值、8-连通域、面积过滤、强度加权质心。

```
lab, n = label(image > threshold, structure=np.ones((3, 3)))
for idx, sl in enumerate(find_objects(lab), start=1):
    mask = lab[sl] == idx
    area = mask.sum()
    if not (min_area <= area <= max_area): continue
    flux = image[sl][mask].sum()
    x = sum((xx + sl[1].start) * w) / sum(w)
    y = sum((yy + sl[0].start) * w) / sum(w)
```

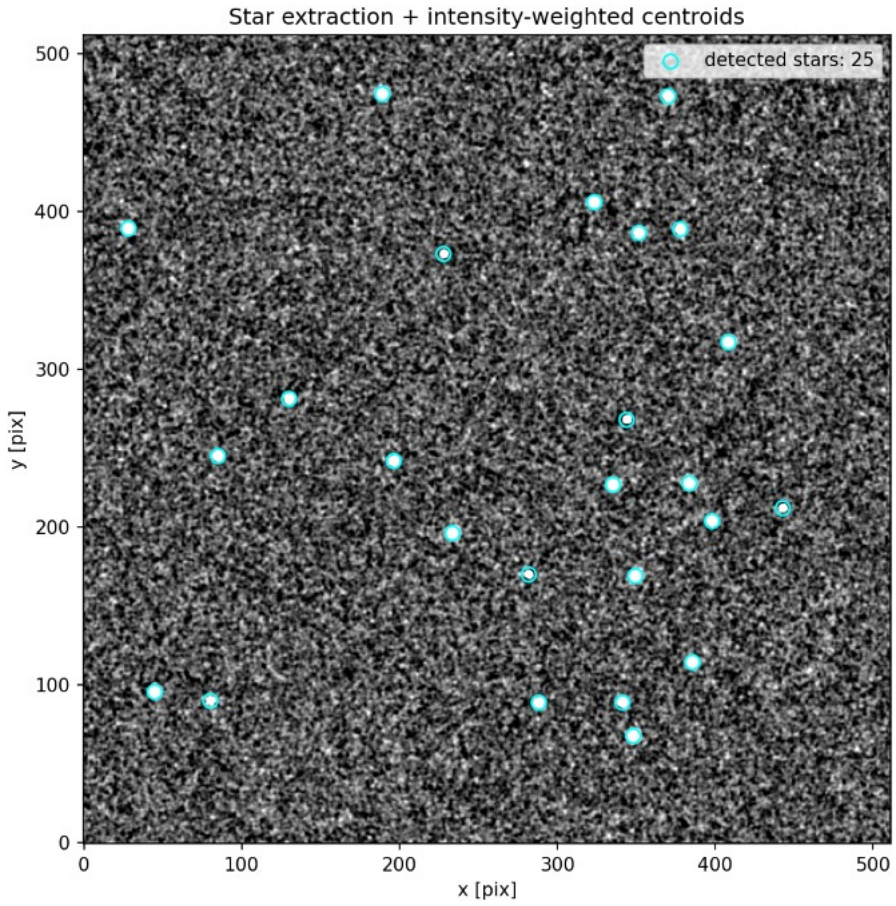


图 5-2　星点提取结果：青色圈是加权质心。

默认 20 颗星，检测到 25 个候选（多出的包含目标/宇宙线/噪声团块）；与星表按 2 pixel 匹配，完备率 1.000，质心 RMSE 0.674 pixel（含 0.6 pixel 指向抖动）。

5.4 运动目标检测

配准用“高斯高通 + 互相关 + 抛物线亚像素”；纯相位相关在 512^2 星场上会锁到噪声峰，这是实际修过的 bug。中值栈作为静止星场模板，残差经 σ=1 pixel 平滑、3σ 阈值、3×3 开/闭运算，再连通域 + Hough + 最近邻轨迹（≥5 帧）确认。

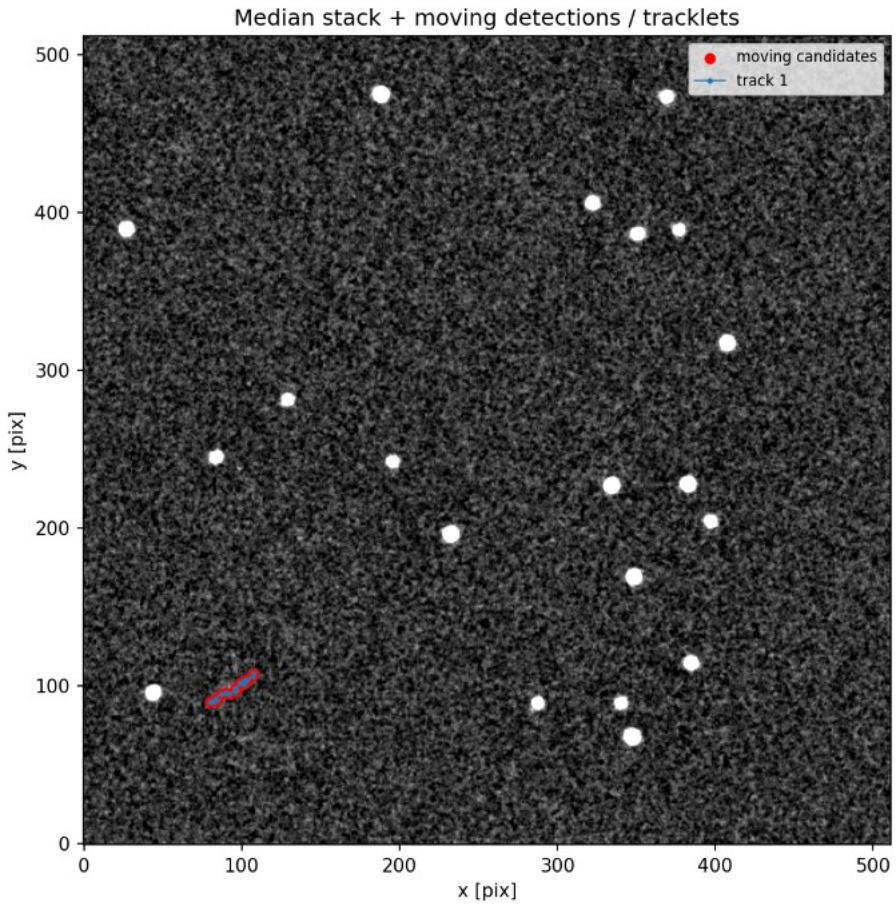


图 5-3　中值栈上的运动目标轨迹（红色候选 + 蓝色轨迹）。

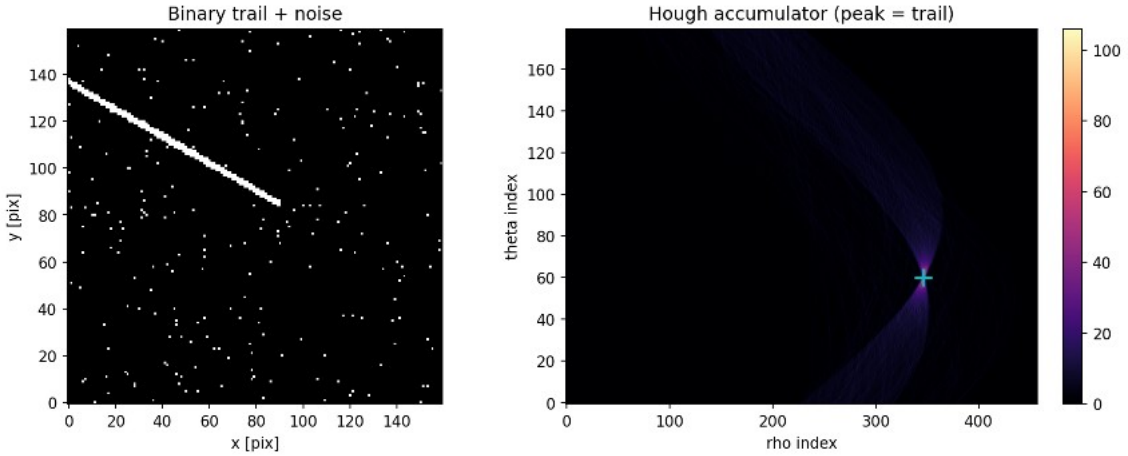


图 5-4　左：带噪声的直线 trail；右：Hough 累加器，青色十字为峰值。

表 5-2　CCD 管线检测结果（仿真）

指标	结果	说明
目标检测率	0.850（17/20）	轨迹确认后；漏检 3 帧
虚警率	0.000（0/17）	单帧候选 98 个，轨迹确认后 0 个假点
目标位置 RMSE	1.094 pixel	含质心偏差和 0.6 pixel 抖动
最长轨迹	17 帧	1 条目标轨迹

5.5 天体测量

TAN WCS 给初始像素→天球映射；三次多项式拟合“探测器→理想像素”的畸变；球面单位向量 + KD-tree 做星表匹配，迭代重拟合。默认 20 颗星全部匹配，星表残差 0.080 arcsec，目标天体测量 RMSE 1.393 arcsec。

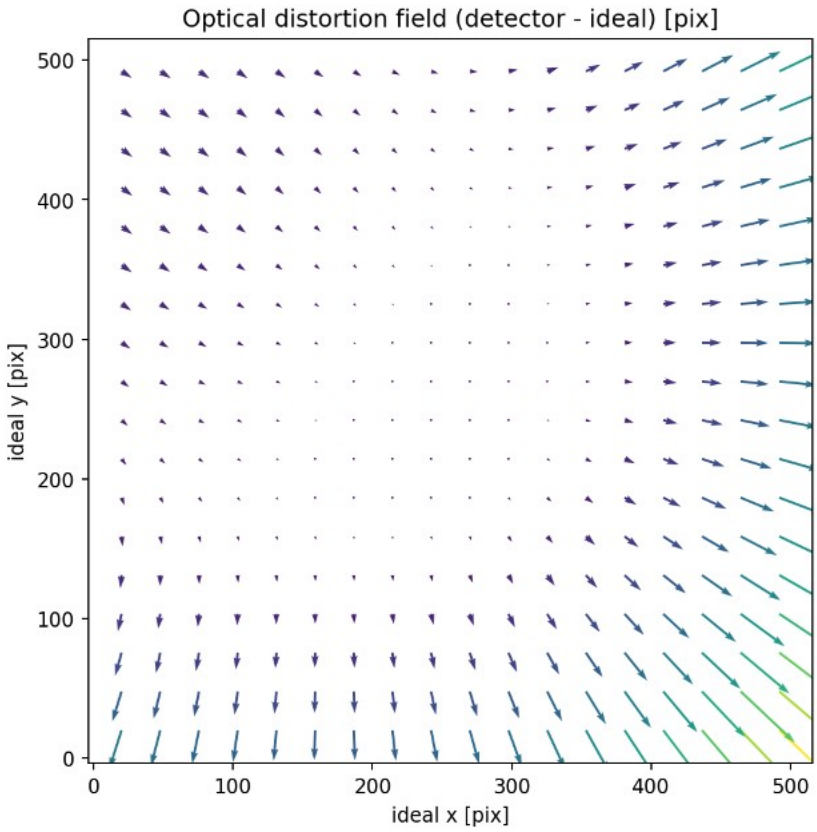


图 5-5　探测器相对理想像素的畸变场：中心小、边缘大。

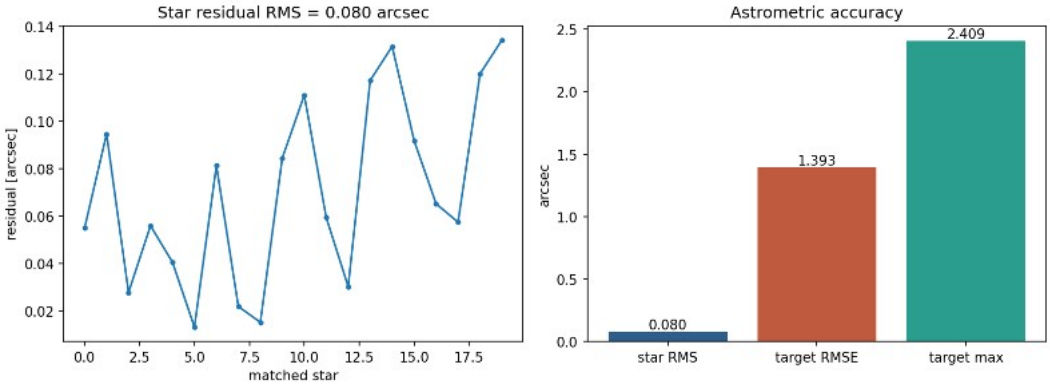


图 5-6　左：20 颗匹配星残差；右：星表/目标/最大误差。

5.6 评估与可视化

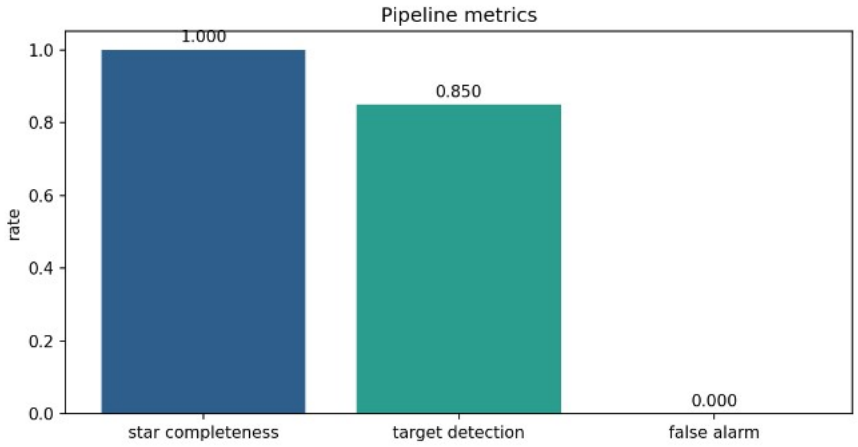


图 5-7　星点完备率、目标检测率、虚警率。

表 5-3　CCD 管线端到端结果（仿真）

指标	结果	口径
星点完备率	1.000	20/20，2 pixel 半径
星点质心 RMSE	0.674 pixel	含 0.6 pixel 抖动
目标检测率	0.850	17/20 帧，3 pixel 半径
虚警率	0.000	轨迹确认后 0/17
目标位置 RMSE	1.094 pixel	匹配点
星表残差	0.080 arcsec	3 arcsec 门限
目标天体测量 RMSE	1.393 arcsec	RA/Dec 角距
运行时间/内存	7.01 s / 312 MB	CPU，20 帧 512²

06 学习型相位反演

CNN 给初值、BOBYQA 精修；重点是用评估次数证明省在哪。

6.1 可微前向模型

ml/phase_torch.py 用 PyTorch 实现 64×64 的相位差前向：圆孔光瞳、Z4/Z8/Z9、5 帧 d=[−2,−1,0,1,2] λ、FFT、2×2 binning。所有操作可微，可生成训练数据，也可作为 unrolled 优化的前向。

代码 6-1　可微前向的核心；与主项目共享物理，但网格/星点不同。

```
mask, z4, z8, z9 = zernike_basis(fft_n, pupil_radius)
phase = 2*pi*(c4*z4 + c8*z8 + c9*z9 + d*z4)
pupil = mask * exp(1j*phase)
psf = |fftshift(fft2(pupil))|^2
psf = avg_pool2d(psf, oversample)
```

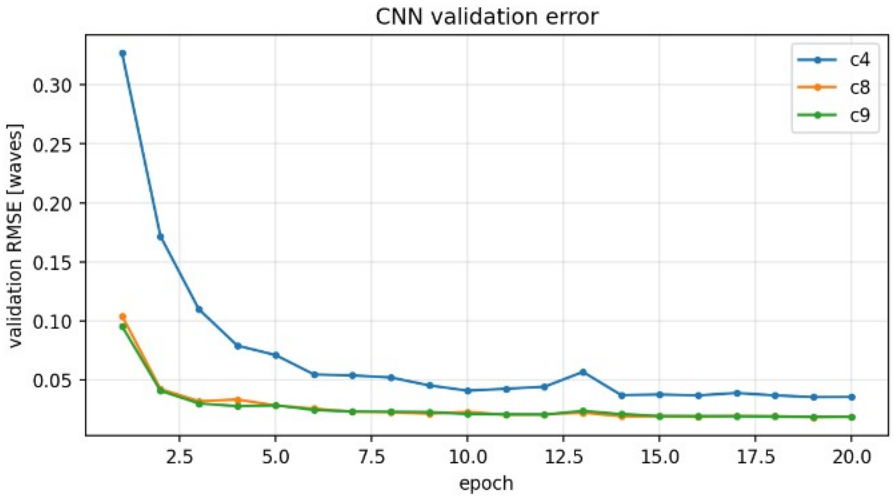


图 6-1　CNN 验证误差随 epoch 下降：c8/c9 约 0.018 waves，c4 约 0.035 waves。

6.2 CNN 结构

表 6-1　ZernikeCNN（约 15 万参数）

层	配置	输出
输入	5×64×64（5 帧）	—
Conv+ReLU+Pool	16/32/64 通道，3×3	64×8×8
Pool	AdaptiveAvgPool2d(4)	64×4×4
Head	Flatten→128→3	(c4, c8, c9)

训练：2048 个在线样本、batch 64、Adam 1e-3、CosineAnnealing、20 epochs；输入按帧峰值归一化，标签按量程归一化。噪声是每帧峰值的 1% 高斯；直接按 1.0 加噪声会淹没归一化 PSF，这是实际修过的 bug。

6.3 two-stage：34 vs 1823

固定同一个 NumPy 正向模型和同一个 NLopt `LN_BOBYQA`，只换初值：A 是 CNN 一次前向 + 一次 BOBYQA；B 是 40 个随机起点，每个跑一次 BOBYQA，取最好。

表 6-2　3 个独立测试样本的 two-stage 结果（仿真）

方法	平均评估次数	平均时间	波前 RMS 误差
CNN + BOBYQA	34	0.226 s	0.00955
40 随机 + BOBYQA	1823	13.33 s	0.00955

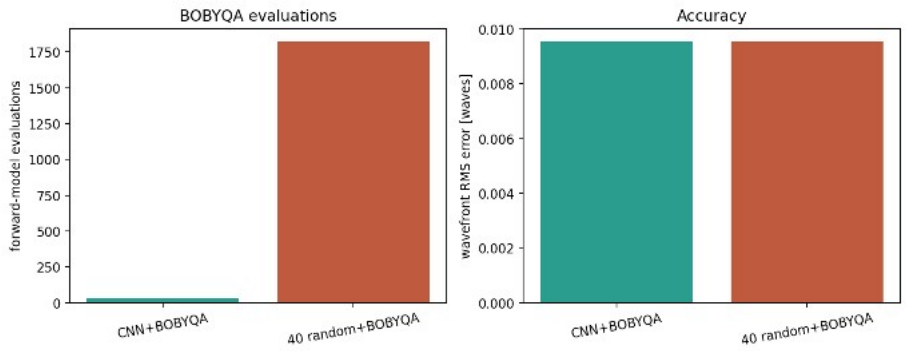


图 6-2　CNN 初值与 40 随机起点的 BOBYQA 评估次数/精度对比。

口径提醒

这只是方法验证：BOBYQA maxeval=120，baseline 是“40 起点 × BOBYQA”，不是主报告的“40 起点坐标下降 + 前 3 名 BOBYQA（maxeval=2000）”。34 vs 1823 说明“好初值省评估”，不能写成主项目已经节省 54×。

6.4 与主报告的关系

表 6-3　主求解器与 ML 代理的差异（不能混表）

项	主报告	本 ML 实验
网格	256×256，过采样 2（FFT 512²）	64×64，过采样 2
星点	15 颗，星点叠加	纯 PSF，无星点
帧数/变量	5 帧 / Z4/Z8/Z9	5 帧 / 3 项
优化	40 起点 + 坐标下降 + top3 BOBYQA，maxeval=2000	CNN 初值 + BOBYQA maxeval=120
结果	波前 RMS 0.001667 λ	波前 RMS 误差 0.00955（方法验证）

6.5 测试

`ml/tests/test_phase.py` 4 项：前向形状/归一化、梯度非零、波前 RMS、数据集标签归一化；全部通过。训练/推理脚本见 `ml/train_cnn.py`、`ml/compare.py`、`ml/two_stage.py`。

07 FastAPI + React 科学计算小程序

> 把仿真/反演做成可交互页面，而不是只留命令行。

7.1 功能

- 滑杆调 c4/c8/c9/noise，生成 5 帧离焦 PSF；
- 一键反演：CNN 初值 + BOBYQA，Plotly 对比 truth/CNN/refined；
- 显示评估次数、最终 cost、模型是否加载。

7.2 后端 API

表 7-1 FastAPI 接口

方法/路径	输入	输出
`GET /api/health`	—	服务/模型状态
`POST /api/simulate`	c4,c8,c9,noise	5 帧 PNG data URL、shape
`POST /api/invert`	c4,c8,c9,noise	truth/CNN/refined/nfev/cost/shape
`GET /api/metrics`	—	CCD 管线 metrics.json
`GET /api/figures`	—	结果图列表
`GET /`	—	React 单页

代码 7-1 反演接口：CNN 用峰值归一化，BOBYQA 用 sum 归一化。

```
@app.post("/api/invert")
def invert(req: SimRequest):
    stack = forward_stack(torch.tensor(truth), cfg, add_noise=True).numpy()
    cnn = model(peak_normalise(stack)) * scales
    x, nfev, cost = bobyqa(cnn, sum_normalise(stack))
    return {"truth": truth.tolist(), "cnn": cnn.tolist(),
            "refined": x.tolist(), "nfev": nfev, "cost": cost}
```

7.3 前端

React 18 UMD + Babel Standalone（浏览器内 JSX）+ Plotly.js CDN，单文件 `web/ui/index.html`，不需要 npm 构建。左栏参数/按钮，右栏 5 帧缩略图、指标卡、Plotly 柱状图。

代码 7-2　启动与接口自测。

```
python3 -m uvicorn web.api.main:app --reload --port 8000
# http://127.0.0.1:8000/
python3 web/test_api.py   # health/simulate/invert/index
```

实测：`health` 返回 `{"ok":true,"model":true}`；`simulate` 返回 5 帧 [5,64,64]；`invert` 在 truth=[0.6,−0.2,0.1] 上 refined=[0.596,−0.220,0.111]，nfev≈30；`/` 返回 200。

小程序边界

页面是 64×64 PSF 仿真/反演，不是完整 256²/15 星 CCD 管线；完整管线仍用 `pipeline.run`。React/Plotly 走 CDN，离线部署需把 vendor 文件本地化。

08 测试、CI 与性能

> 每一层都有独立的验证，CI 只跑能稳定复现的部分。

8.1 本地测试矩阵

表 8-1　验证矩阵

测试	数量	命令	结果
主项目 CTest	3	`ctest --test-dir build`	3/3 passed
数值库 GTest	10	`ctest --test-dir build-numerics`	10/10 passed
ASan 构建	1	`ctest --test-dir build-numerics-asan`	1/1 passed
pipeline unittest	8	`python3 -m unittest discover -s pipeline/tests`	8/8 passed
ML unittest	4	`python3 -m unittest discover -s ml/tests`	4/4 passed
FastAPI 接口	3	`python3 web/test_api.py`	health/simulate/invert 200

8.2 CI

`.github/workflows/ci.yml` 两个 job：`cpu-and-pipeline`（CPU 构建 + CTest + Python pipeline）和 `numerics`（Eigen/GTest/Benchmark/pybind11 + GTest + smoke test）。当前 `engineering-mvp` 分支两次运行均 success，badge 为 passing。

代码 8-1　CI 结构；本地等价命令见附录 A。

```
name: ci
on: [push, pull_request]
jobs:
  cpu-and-pipeline: ...
  numerics: ...
```

8.3 性能

数值库 benchmark 见表 4-4：solve 128 约 0.57 ms，FFT 4096 约 67 μs，符合 $O(N^3)$/$O(N \log N)$。主项目 v2.1 的 45° 反演约 5692 次正向评估、7.35 s、38.5 MB 显存；CUDA 优化点见报告/旧文档，本文不重复。

8.4 Docker

Dockerfile 已提交（Ubuntu 20.04 + numerics + 主项目 CPU + CTest）。本机 Docker Hub 访问超时，未完成镜像构建；失败原因/复现命令见 `docs/Docker 构建记录.md`。不要把“未构建”写成“已验证”。

09 运行说明、输出与代码注释

> 把命令、文件、注释位置写清楚，便于复现和交接。

9.1 运行命令

代码 9-1　一键复现命令。

```
cd telescope_gpu

# 主项目（CPU/GPU）
cmake -S . -B build
cmake --build build -j4
ctest --test-dir build --output-on-failure

# 数值库
cmake -S numerics -B build-numerics -DCMAKE_BUILD_TYPE=Release
cmake --build build-numerics -j4
ctest --test-dir build-numerics --output-on-failure

# CCD 管线
python3 -m pipeline.run --config configs/pipeline.json --force-sim

# ML
python3 -m ml.train_cnn --epochs 20 --samples 2048 --batch 64 --n 64
python3 -m ml.two_stage --samples 3 --starts 40 --maxeval 120

# Web
python3 -m uvicorn web.api.main:app --reload --port 8000
```

9.2 输出文件

表 9-1　主要输出

输出	内容
`star_field_*.fits`	主项目 5 帧 FITS + 星表头
`inverse_result.txt` / `inverse_report.txt`	主项目系数/诊断
`results/ccd_pipeline/metrics.json`	CCD 检测率/虚警率/位置/天体测量
`results/ccd_pipeline/fig_*.png`	数据/星点/轨迹/残差/指标图
`ml/runs/cnn.pt`	CNN 权重 + scales + 数据配置
`ml/results/two_stage.csv`	two-stage 评估次数/时间/误差
`docs/工程化技术说明_代码对齐版.docx`	本文

9.3 关键配置

表 9-2　关键配置（不同模块不可混用）

配置	值	说明
主项目：帧/变量	5 帧 / Z4/Z8/Z9	0、±1、±2 λ
主项目：起点/精修	40 / 3	坐标下降 + BOBYQA
主项目：maxeval/ftol/xtol	2000 / 1e-10 / 1e-6	报告表 3-1
CCD：图像/星数	512×512 / 20	演示数据集
CCD：目标流量/噪声	1200 e- / 5 e- RMS	仿真
ML：网格/帧数	64×64 / 5	方法验证
ML：CNN 训练	2048 样本 / 20 epochs	CPU

9.4 代码注释与 docstring

表 9-3　注释约定与索引

代码	注释方式	索引
`include/telescope/*.hpp`、`src/*.cpp/.cu`	文件头职责 + 函数前参数/返回/异常；kernel 线程映射	`docs/代码模块注释索引.md` §1
`pipeline/*.py`	模块 docstring（输入/输出/算法）+ 公开函数参数/返回	§2
`numerics/*`	Doxygen `@file`/函数注释 + GTest 目的	§3
`ml/*`	模块 docstring + 训练/接口注释	§3
`web/*`	FastAPI 端点 docstring/注释；React 组件注释	§3/§5

注释原则：文件头写“这个模块解决什么问题”；函数前写“输入/输出/异常/单位”；算法步骤解释“为什么”；不要复述代码字面意思。

A 附录：与主报告逐项对齐清单

> 给评审/答辩用的一页纸。

表 A-1　代码/报告对齐清单

检查项	主报告	本工程	结论
帧数	5（0、±1、±2 λ）	主程序 config + ML 5 帧	一致
变量	Z4/Z8/Z9	主程序 + ML 3 项	一致
代价	图像域归一化 SSE	`phase_diversity.cu`	一致
优化	40 起点 + 坐标下降 + top3 BOBYQA	config 40/3；ML 仅方法验证	主程序一致
BOBYQA 参数	maxeval 2000、ftol 1e-10、xtol 1e-6	config 一致；ML 120 仅实验	主程序一致
灵敏度矩阵	3×5，外部标定、代码未实现	报告表 2-6；代码无 IM	一致
主结果	45° RMS 0.001667 λ、5692 次	报告表 4-1	一致
CCD 管线	报告未展开	独立 pipeline	补充，不混表
ML 结果	报告未展开	64×64 代理，34/1823	方法验证
实测	无	无	全部仿真

A.1 复现命令

代码 A-1　复现命令。

```
# 主项目
cmake -S . -B build && cmake --build build -j4
ctest --test-dir build --output-on-failure

# 数值库
cmake -S numerics -B build-numerics -DCMAKE_BUILD_TYPE=Release
cmake --build build-numerics -j4
ctest --test-dir build-numerics --output-on-failure

# CCD / ML
python3 -m pipeline.run --config configs/pipeline.json --force-sim
python3 -m ml.train_cnn --epochs 20 --samples 2048 --batch 64 --n 64
python3 -m ml.two_stage --samples 3 --starts 40 --maxeval 120

# Web
python3 -m uvicorn web.api.main:app --port 8000
```

A.2 术语

表 A-2　术语表

术语	含义
相位差	多帧已知离焦，靠多样性反演波前
Zernike	圆域正交多项式；本项目的 Z9 ≠ Noll Z9
SSE	图像域平方误差
BOBYQA	无梯度二次模型信赖域优化
WCS/TAN	世界坐标系统/切平面投影
two-stage	CNN 初值 + 传统精修

配套文件：docs/代码模块注释索引.md、docs/Docker 构建记录.md、pipeline/README.md、numerics/README.md、ml/README.md、web/README.md。